

В Linux действа принципът, че всичко е файл. Основните атрибути на файловете са:

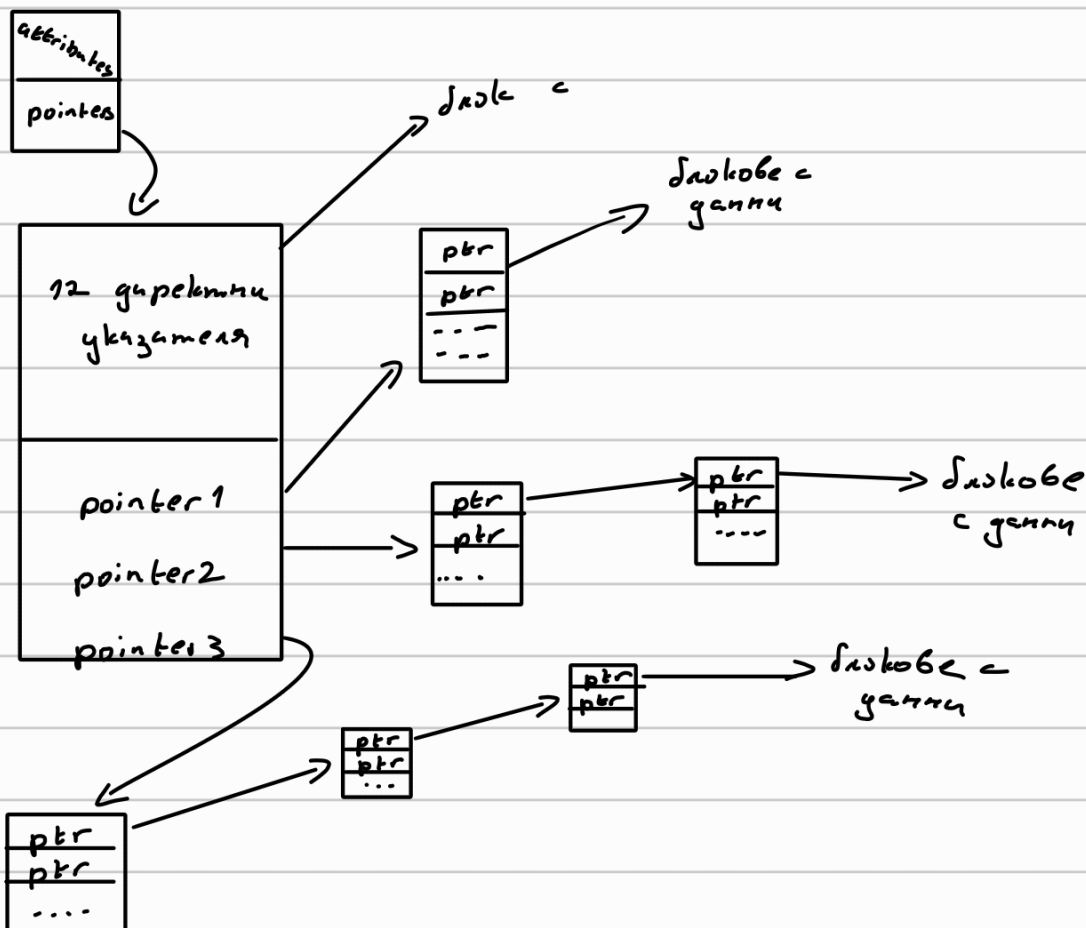
- размер
- тип и права за достъп (например `drwxrwxrwx`)
- собственик (UID и GID)
- време на последна промяна, време на последен достъп, ...
- име

Всички атрибути с изключение на името се пазят в `inode` на този.

Името се пазят в директорията, която е списък от двойки `символ, номер inode`, т.е. `{символ, номер на inode}, ..., {символ, номер на inode}` *

Всеки `inode` пазят:

- атрибутите на файла
- 12 директни указателя към блокове с данни
- 3 индиректни указателя (по 3 числа)



* Hardlinks са само нови entry в този лист с ново име, но със същия номер на `inode`. Невозможни са от оригиналното entry

Права на всеки обект се пазят като 12 бита.

• Девет основни бита (по 3 за всяко от източниците)

1. owner - собственика на файл

2. group - потребителите от групата

3. other - всички останали

За всяка категория има 3 права:

r - read

w - write

x - execute

Например (-rw-rw-r--) е файл който собственикът и потребителите от групата могат да четат и променят, а всички други само да четат

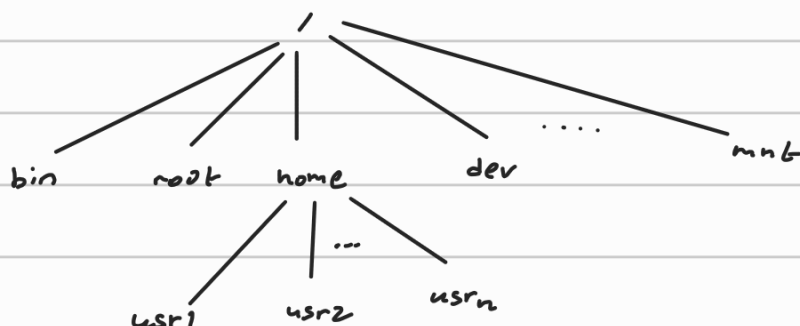
• 3 специални бита

1 за setuid - изпълнен файл работи с права на собственика си, а не с права на този който го изпълнява

1 за setgid - аналогично на setuid но за групата

1 sticky bit - върху директория значи че всеки може да създава файлове, но може да прече само своите файлове

Единно пространство от именувани обекти е кореново дърво с корен /



Дискът е разделен на сектори - най-малката единица, която хардуерът може да четем/пишем наведнъж, но файловата система разделя диска на блокове всеки от които по размер могат да си няколко сектора.

- Зареждане на блок - кой за какво зареждане на ОС
- суперблок - метагиги за файловата система
- модул за `inodes`
- Блокове с групи

Точки на контуране

Можем да контролираме улога фајлова система тем сезитател в дадена точка на контриране (директорие от съществуващото дърво. По-малко уружати фс също е кореново дърво след контриране ползване от по-малко кореново дърво. Монтиране с mount и демонтиране с umount

Бүгдє оѣкту:

- обикновени файлове
- директории - специални файлове с двойки скоби, номер на inode
- устройства - блокови (disk) и символни (терминал), като файлове в /dev
- информационни канали - изкубвани тръби (fifo) и асинхронни тръби (pipe)
- символни връзки - съвършат път до целта

Ускоряване на достъпа

Докът се генерира на дякобе и дурото нази кония на пастору
ползватите дякобе в паметта. Така при зетене се проверява дали
докът е в кеша и ако е се връща директно от паметта. При писане
се променя копие, което е в паметта, а реалният запис върху диска
се отлага. По-дълго е, но може да забележи зазуде на данни при
спиране на мока компилер.

Забвките към гуска не се обвързват в реда на познанията, а се подреждат по познания на паметите. Така се минимизира губещият ефект на забвенията

Журналична файлова система

Преди промени да се направят върху диска се записват в журнала като транзакции. Всяка транзакция завършва с commit маркер, който я обявява за валидна. При стартиране след авария се презарежда журнала и завършените транзакции се правят като върху диска, а незавършените се отхвърлят.

RAID0 - Поддържа се само едно копие на 2 диска. Ако един се повреди другите имат само едно копие. При 2 диска само 50% са използвани, при 3 диска - 33,33%, при N диска - $\frac{1}{N}$

RAID5 - Изисква се поне 3 диска. Поддържа се паритет по който при повреда на 1 диск информацията може да се възстанови от другите. Паритетът се съхранява с XOR. Пример с 3 диска:

Disk 1	Disk 2	Disk 3	Използвана част: 66,67%
A_1	A_2	$P = A_1 \oplus A_2$	
B_1	$P = B_1 \oplus B_3$	B_3	
$P = C_1 \oplus C_3$	C_2	C_3	

open() - отваря/създава файл. Връща файлов дескриптор и -1 при грешка
close() - затваря файл и освобождава дескриптора, връща 0 при успех и -1 при error
read() - чете до n байта и връща брой прочетени (0 = край на файла, -1 = грешка)
и отменява позицията с брой прочетени
write() - чете до n байта и връща брой записани или -1 при грешка и отменява позицията с брой записани
lseek() - премества текущата позиция на позиция получена от дадено начало (начало, сегашна позиция, край) с дадено отместване. Не работи върху socket-и и пайпи